Unidad 2:
Scripts de Administración y Automatización en el SGBD

FICHA TÉCNICA DE LA UNIDAD

Unidad Trabajo	UD 2: Scripts de Administración y Automatización en el SGBD
Resultados de Aprendizaje Asociados	**RA3:** Automatiza tareas de administración del sistema gestor, realizando guiones y programando tareas.
Criterios de Evaluación	CE 3.a, CE 3.b, CE 3.c, CE 3.d
Tiempo	12 horas lectivas (equivalente a unas 4 semanas de clase en el módulo ASGBD)

ÍNDICE DE CONTENIDOS

1. Repaso Teórico: Programación en el Servidor (MySQL)

1.1. Directivas Básicas y Tipos de Variables

1.2. Procedimientos Almacenados (Stored Procedures)

1.3. Funciones Almacenadas (Stored Functions)

1.4. Estructuras de Control de Flujo (Condicionales y Bucles)

1.5. Cursores (Procesamiento Secuencial de Filas)

1.6. Control de Excepciones (Handlers) y Transacciones

1.7. Disparadores (Triggers)

1.8. Casos Prácticos de Mantenimiento y Administración mediante Rutinas y Triggers

2. Ventajas de la programación en la administración

3. Catálogo del Sistema (Metadatos)

3.1. information_schema (El Diccionario de Datos)

3.2. Base de Datos mysql (El Repositorio de Seguridad y Privilegios)

3.3. Base de Datos performance_schema

3.4. Base de Datos sys (La Abstracción Simplificada)

4. Automatización Mediante Scripts Externos (Bash)

5. Ejemplos Prácticos de Scripts de Administración (Sencillos y Documentados)

1. Repaso Teórico: Programación en el Servidor (MySQL)

La administración moderna de bases de datos no se limita a ejecutar consultas interactivas[cite: 45]. Para garantizar la eficiencia, seguridad y automatización, los SGBD permiten compilar y ejecutar lógica programada directamente en el servidor a través de procedimientos, funciones, cursores y disparadores[cite: 2, 45].

1.1. Directivas Básicas y Tipos de Variables

- **Delimitador de Sentencias (`DELIMITER`):** En MySQL, el terminador por defecto es el punto y coma (`;`)[cite: 3]. Al programar bloques de código que contienen múltiples instrucciones, es obligatorio cambiar temporalmente este delimitador (comúnmente por `$$` o `//`) para evitar que el intérprete corte la rutina antes de tiempo[cite: 3].
- **Variables Locales (`DECLARE`):** Se definen exclusivamente dentro de bloques `BEGIN ... END` de procedimientos, funciones o disparadores[cite: 5, 6]. Su ámbito finaliza al concluir el bloque[cite: 5, 6]. *Regla obligatoria de orden:* deben declararse siempre al inicio del bloque, antes que los cursores y manejadores de error (`HANDLER`)[cite: 6].
- **Variables de Sesión de Usuario:** Llevan como prefijo la arroba (`@nombre`)[cite: 5]. No requieren declaración explícita de tipo de dato y su valor perdura durante toda la sesión o conexión activa del cliente con el servidor[cite: 5].

1.2. Procedimientos Almacenados (Stored Procedures)

Son subprogramas que se compilan y almacenan en el SGBD[cite: 45]. Pueden realizar operaciones complejas, alterar datos en múltiples tablas y ejecutar sentencias DDL (Data Definition Language)[cite: 45].

- **Parámetros:**[cite: 45]
  - `IN` (Entrada): El valor se pasa al procedimiento pero no se puede modificar hacia el exterior (paso por valor)[cite: 4, 45].
  - `OUT` (Salida): El procedimiento asigna un valor a este parámetro para devolverlo al entorno que lo invocó (paso por referencia)[cite: 4, 45].
  - `INOUT` (Entrada/Salida): El parámetro aporta un valor inicial y puede ser modificado dentro del procedimiento para devolver el nuevo estado resultante[cite: 4, 45].
- **Invocación:** Se ejecutan explícitamente mediante la instrucción `CALL nombre_procedimiento(parámetros);` [cite: 2, 5, 45].
- **Asignación de datos:** Para volcar resultados SQL en variables se utiliza tanto `SET variable = (SELECT ...)` como la cláusula directa `SELECT col INTO variable FROM ...` [cite: 8].
- **Uso Administrativo:** Ideales para tareas de mantenimiento repetitivas (ej. creación masiva de usuarios, particionamiento de tablas, purgas programadas de logs)[cite: 45].

Ejemplo de Sintaxis Básica (MySQL)[cite: 45]

```
DELIMITER //
CREATE PROCEDURE sp_crear_usuario_lectura(
IN p_username VARCHAR(50),
IN p_password VARCHAR(50),
IN p_db VARCHAR(50)
)
BEGIN
  -- Generar la sentencia dinámicamente y ejecutarla de forma segura
  SET @sql_create = CONCAT('CREATE USER "', p_username, '"@"localhost" IDENTIFIED BY "', p_
  PREPARE stmt1 FROM @sql_create;
  EXECUTE stmt1;
  DEALLOCATE PREPARE stmt1;
  SET @sql_grant = CONCAT('GRANT SELECT ON ', p_db, '.* TO "',p_username, '"@"localhost";')
  PREPARE stmt2 FROM @sql_grant;
  EXECUTE stmt2;
```

```
  DEALLOCATE PREPARE stmt2;
  FLUSH PRIVILEGES;
END //
DELIMITER ;
```

1.3. Funciones Almacenadas (Stored Functions)

A diferencia de los procedimientos, las funciones están diseñadas específicamente para **calcular y devolver un único valor escalar** (un número, una cadena de texto, una fecha) [cite: 45].

- **Características Clave:**[cite: 45]
  - Todos los parámetros recibidos son obligatoriamente de **entrada** (no se coloca la palabra `IN`)[cite: 11].
  - Deben especificar el tipo de retorno en su cabecera mediante `RETURNS tipo_dato` y finalizar con una instrucción `RETURN valor;` [cite: 10, 11, 45].
  - **Restricción Binaria / Determinismo:** Es obligatorio definir al menos una de sus características de comportamiento: `DETERMINISTIC` (mismos datos producen siempre el mismo resultado), `NO SQL` o `READS SQL DATA` (para evitar el error 1418 en servidores con binary logging activado)[cite: 12]. Alternativamente, el administrador puede fijar en el servidor `SET GLOBAL log_bin_trust_function_creators = 1;` [cite: 12].
  - **No** pueden invocar sentencias de transacción (COMMIT o ROLLBACK) ni modificar directamente el estado de las tablas en la mayoría de escenarios de consulta estándar[cite: 45].
  - Se integran e invocan directamente dentro de sentencias SQL estándar (`SELECT funcion(...)` , `WHERE` , `ORDER BY`)[cite: 2, 45].

Ejemplo de Sintaxis Básica (MySQL)[cite: 45]

```
DELIMITER //
CREATE FUNCTION fn_obtener_tamano_megas(p_bytes BIGINT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
```

```
  RETURN p_bytes / 1024 / 1024;
END //
DELIMITER ;

-- Uso de la función en una consulta administrativa:
SELECT table_name, fn_obtener_tamano_megas(data_length) AS 'Tamano_MB'
FROM information_schema.tables
WHERE table_schema = 'mi_base_datos';
```

1.4. Estructuras de Control de Flujo (Condicionales y Bucles)

Permiten alterar el flujo secuencial de ejecución dentro de rutinas almacenadas[cite: 13, 14, 15, 16]:

- **Condicionales:**
  - `IF ... THEN ... [ELSEIF ... THEN ...] [ELSE ...] END IF;` [cite: 13]
  - `CASE ... WHEN ... THEN ... [ELSE ...] END CASE;` (admite evaluación de valor directo o de expresiones booleanas)[cite: 14].
- **Bucles e Iteraciones:**
  - `WHILE condición DO ... END WHILE;` (evalúa la condición antes de cada iteración) [cite: 16].
  - `REPEAT ... UNTIL condición END REPEAT;` (ejecuta el bloque al menos una vez hasta que la condición se haga verdadera)[cite: 15, 38].
  - `LOOP ... END LOOP;` (bucle continuo con etiquetas; requiere `LEAVE etiqueta;` para salir o `ITERATE etiqueta;` para pasar a la siguiente iteración)[cite: 14].

1.5. Cursores (Procesamiento Secuencial de Filas)

Los cursores permiten recorrer fila a fila el conjunto de resultados generado por una consulta `SELECT` [cite: 21]. En MySQL son estrictamente de **sólo lectura** (*read-only*) y **unidireccionales** (*non-scrollable*, únicamente avanzan hacia delante)[cite: 21].

Siguen de forma obligatoria cuatro pasos secuenciales[cite: 21, 22]:

1. `DECLARE cursor_name CURSOR FOR sentencia_select;` (se declaran después de las variables y antes de los handlers)[cite: 21].
2. `OPEN cursor_name;` (ejecuta la consulta e inicializa el puntero)[cite: 21, 22].
3. `FETCH cursor_name INTO var1, var2, ...;` (recupera la fila actual en variables y avanza)[cite: 22].
4. `CLOSE cursor_name;` (libera los recursos de memoria asignados)[cite: 22].

Para controlar cuándo se han terminado de leer todas las filas, se asocia un manejador de tipo `NOT FOUND` (código SQLSTATE '02000')[cite: 18, 22]:

```
DECLARE fin INT DEFAULT FALSE;
DECLARE cur CURSOR FOR SELECT id, nombre FROM usuarios;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;

OPEN cur;
bucle_lectura: LOOP
  FETCH cur INTO v_id, v_nombre;
  IF fin THEN
    LEAVE bucle_lectura;
  END IF;
  -- Procesamiento individual de cada fila...
END LOOP;
CLOSE cur;
```

1.6. Control de Excepciones (Handlers) y Transacciones

MySQL permite atrapar errores producidos durante la ejecución mediante `DECLARE ... HANDLER` [cite: 17]:

- **Acción del manejador:** `CONTINUE` (ejecuta el handler y sigue con la siguiente instrucción) o `EXIT` (ejecuta el handler y aborta el bloque `BEGIN...END` actual)[cite: 17].
- **Condición capturada:** Puede indicarse un código de error numérico de MySQL (ej. 1062 por clave primaria duplicada), un valor `SQLSTATE` (ej. '23000'), o clasificadores generales como `SQLEXCEPTION` o `SQLWARNING` [cite: 17, 18, 41].
- **Control transaccional:** La combinación de handlers con transacciones permite asegurar la atomicidad de operaciones críticas en administración[cite: 20]:

```
DELIMITER $$ CREATE PROCEDURE sp_operacion_segura() BEGIN -- Si ocurre cualquier excepción $
DELIMITER ;
```

1.7. Disparadores (Triggers)

Un disparador es un bloque de código que se ejecuta **de manera automática** en respuesta a un evento DML (INSERT, UPDATE o DELETE) sobre una tabla específica[cite: 2, 24, 25, 45].

- **Estructura temporal:**[cite: 45]
  - `BEFORE`: Se ejecuta antes de que los cambios se validen e inserten en la base de datos (útil para validación, normalización de campos o saneamiento de datos)[cite: 25, 45].
  - `AFTER`: Se ejecuta después de que el cambio se ha confirmado en la tabla (ideal para auditorías, propagación de datos y sincronizaciones)[cite: 45].
- **Variables Especiales:**[cite: 45]
  - `OLD`: Accede a los valores originales de la fila antes del evento (disponible en `UPDATE` y `DELETE`)[cite: 45].
  - `NEW`: Accede a los nuevos valores entrantes (disponible en `INSERT` y `UPDATE`)[cite: 45]. En triggers de tipo `BEFORE`, permite reasignar directamente el valor asignando a `NEW.columna = ...` [cite: 26].
- **Uso Administrativo:** Monitorización de cambios críticos, auditoría de seguridad (saber quién modificó qué y cuándo) e integridad referencial compleja[cite: 45].

Ejemplo de Sintaxis de Auditoría (MySQL)[cite: 45]

```
CREATE TABLE log_cambios_privilegios (
id INT AUTO_INCREMENT PRIMARY KEY,
usuario_afectado VARCHAR(100),
accion VARCHAR(20),
fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
realizado_por VARCHAR(100)
);

DELIMITER //
CREATE TRIGGER tg_auditoria_usuarios_update
```

```
AFTER UPDATE ON mysql.user
FOR EACH ROW
BEGIN
  INSERT INTO log_cambios_privilegios (usuario_afectado, accion, realizado_por)
  VALUES (OLD.user, 'MODIFICACION_PERMISOS', USER());
END //
DELIMITER ;
```

1.8. Casos Prácticos de Mantenimiento y Administración mediante Rutinas y Triggers

En el trabajo cotidiano de un Administrador de Sistemas de Bases de Datos (DBA), los procedimientos, funciones y disparadores no se limitan a cálculos matemáticos; resuelven problemas de optimización física del almacenamiento, saneamiento por lotes y blindaje de seguridad interna:

Caso 1 (Procedimiento): Desfragmentación y Optimización Masiva de Tablas

Tras grandes operaciones de borrado o actualización, el espacio en disco queda fragmentado en los motores de almacenamiento (como InnoDB o MyISAM). Este procedimiento utiliza un cursor sobre el catálogo `information_schema.tables` [cite: 45] y construye consultas preparadas dinámicas (`PREPARE / EXECUTE`) para desfragmentar de forma consecutiva todas las tablas de una base de datos específica:

```
DELIMITER $$DROP PROCEDURE IF EXISTS sp_admin_optimizar_tablas$$
CREATE PROCEDURE sp_admin_optimizar_tablas(IN p_esquema VARCHAR(64))
BEGIN
  DECLARE v_fin INT DEFAULT FALSE;
  DECLARE v_tabla VARCHAR(64);
  -- Cursor que recupera todas las tablas físicas del esquema indicado
  DECLARE cur_tablas CURSOR FOR
    SELECT table_name
    FROM information_schema.tables
    WHERE table_schema = p_esquema AND table_type = 'BASE TABLE';
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;

  OPEN cur_tablas;
  bucle_tablas: LOOP
    FETCH cur_tablas INTO v_tabla;
```

```
    IF v_fin THEN
      LEAVE bucle_tablas;
    END IF;

    -- Construcción y ejecución de la sentencia DDL de mantenimiento
    SET @sql_opt = CONCAT('OPTIMIZE TABLE `', p_esquema, '`.`', v_tabla, '`;');
    PREPARE stmt FROM @sql_opt;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;
  END LOOP;
  CLOSE cur_tablas;
END $$ DELIMITER ;  -- Invocación para optimizar todas las tablas de la BD de producción: --
```

Caso 2 (Procedimiento): Purga Masiva Segura sin Bloqueo de Transacciones

Ejecutar un `DELETE FROM logs WHERE fecha < ...` directo sobre tablas con millones de filas bloquea los recursos del servidor y llena el buffer del log de transacciones. La buena práctica consiste en realizar el borrado en **lotes acotados (chunks)** mediante un bucle `WHILE` con `LIMIT` y aplicando `COMMIT` tras cada bloque:

```
DELIMITER $$
DROP PROCEDURE IF EXISTS sp_admin_purgar_logs_por_lotes$$ CREATE PROCEDURE sp_admin_purgar_
DELIMITER ;

-- Llamada: purgar registros de más de 90 días en lotes de 5000 filas
-- CALL sp_admin_purgar_logs_por_lotes(90, 5000, @total_borrados);
-- SELECT @total_borrados AS 'Filas eliminadas con éxito';
```

Caso 3 (Procedimiento): Rotación y Archivado Dinámico de Tablas

Para mantener contenidas las tablas de alto volumen (ej. pedidos antiguos), este procedimiento crea dinámicamente una tabla de histórico identificada por año (si no existiera previamente), traslada los datos cerrados y purga la tabla en producción dentro de una transacción atómica:

```
DELIMITER $$DROP PROCEDURE IF EXISTS sp_admin_archivar_historico_anual$$
CREATE PROCEDURE sp_admin_archivar_historico_anual(IN p_anio INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  -- 1. Crear la tabla destino histórica del año correspondiente
  SET @tabla_historica = CONCAT('pedidos_historico_', p_anio);
  SET @sql_crear = CONCAT('CREATE TABLE IF NOT EXISTS `', @tabla_historica, '` LIKE pedidos
  PREPARE stmt1 FROM @sql_crear;
  EXECUTE stmt1;
  DEALLOCATE PREPARE stmt1;

  -- 2. Transacción de volcado y purga atómica
  START TRANSACTION;
    SET @sql_volcar = CONCAT('INSERT INTO `', @tabla_historica, '` SELECT * FROM pedidos WHE
    PREPARE stmt2 FROM @sql_volcar;
    EXECUTE stmt2;
    DEALLOCATE PREPARE stmt2;

    SET @sql_borrar = CONCAT('DELETE FROM pedidos WHERE YEAR(fecha_pedido) = ', p_anio, ';')
    PREPARE stmt3 FROM @sql_borrar;
    EXECUTE stmt3;
    DEALLOCATE PREPARE stmt3;
  COMMIT;
END $$
DELIMITER ;
```

Caso 4 (Función): Diagnóstico de Fragmentación y Espacio No Reclamado

Los administradores necesitan supervisar qué tablas contienen un alto índice de espacio libre residual (*Data Free*) para planificar ventanas de mantenimiento. Esta función consulta el diccionario de datos y devuelve el porcentaje de espacio desperdiciado sobre el tamaño total de la tabla:

```
DELIMITER $$
DROP FUNCTION IF EXISTS fn_admin_ratio_fragmentacion$$
CREATE FUNCTION fn_admin_ratio_fragmentacion(
  p_esquema VARCHAR(64),
  p_tabla VARCHAR(64)
)
RETURNS DECIMAL(5,2)
READS SQL DATA
BEGIN
```

```
  DECLARE v_data_bytes BIGINT;
  DECLARE v_free_bytes BIGINT;
  DECLARE v_ratio DECIMAL(5,2) DEFAULT 0.00;

  SELECT (data_length + index_length), data_free
  INTO v_data_bytes, v_free_bytes
  FROM information_schema.tables
  WHERE table_schema = p_esquema AND table_name = p_tabla;

  -- Si la tabla tiene datos y espacio libre detectado
  IF (v_data_bytes + v_free_bytes) > 0 THEN
    SET v_ratio = (v_free_bytes / (v_data_bytes + v_free_bytes)) * 100;
  END IF;

  RETURN ROUND(v_ratio, 2);
END $$
DELIMITER ;

-- Uso en una consulta de monitorización:
-- SELECT table_name, fn_admin_ratio_fragmentacion('mi_tienda', table_name) AS 'Fragmentacio
-- FROM information_schema.tables
-- WHERE table_schema = 'mi_tienda' AND fn_admin_ratio_fragmentacion('mi_tienda', table_name
```

Caso 5 (Función): Verificación de Existencia de Privilegios Administrativos

Antes de lanzar despliegues automatizados, es recomendable verificar si una cuenta posee permisos específicos. Esta función consulta la tabla física `mysql.user` [cite: 45] y devuelve un booleano confirmando si el usuario dispone o no de privilegios globales de administración (ej. `SUPER` o `Process_priv`):

```
DELIMITER $$
DROP FUNCTION IF EXISTS fn_admin_es_administrador$$
CREATE FUNCTION fn_admin_es_administrador(
  p_usuario VARCHAR(32),
  p_host VARCHAR(60)
)
RETURNS BOOLEAN
READS SQL DATA
BEGIN
  DECLARE v_es_admin INT DEFAULT 0;

  SELECT COUNT(*)
  INTO v_es_admin
```

```
  FROM mysql.user
  WHERE user = p_usuario
    AND host = p_host
    AND (Super_priv = 'Y' OR Process_priv = 'Y' OR Reload_priv = 'Y');

  IF v_es_admin > 0 THEN
    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
END $$
DELIMITER ;
```

Caso 6 (Disparador): Protección de Seguridad contra Borrados Accidentales

Un error grave de administración es lanzar un `DELETE` sin la cláusula `WHERE` adecuada en producción. Este disparador de tipo `BEFORE DELETE` consulta la hora de ejecución o impide el borrado sobre tablas protegidas si quien lo intenta no es el administrador supremo (`root@localhost`), abortando la operación de inmediato con `SIGNAL SQLSTATE` [cite: 45]:

```
DELIMITER $$
DROP TRIGGER IF EXISTS tg_admin_bloqueo_borrado_critico$$
CREATE TRIGGER tg_admin_bloqueo_borrado_critico
BEFORE DELETE ON configuracion_global
FOR EACH ROW
BEGIN
  -- Validar si el usuario actual es el administrador de mantenimiento
  IF USER() != 'root@localhost' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'ACCESO DENEGADO: La tabla de configuracion solo puede ser modifica
  END IF;

  -- Comprobar si se intenta borrar un parámetro marcado como bloqueado
  IF OLD.bloqueado = 1 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'OPERACION ABORTADA: No se permite la eliminacion de parametros del
  END IF;
END $$
DELIMITER ;
```

2. Ventajas de la programación en la administración

El uso de estos elementos dentro del SGBD ofrece tres ventajas operativas críticas para un Administrador de Sistemas de Bases de Datos (ASGBD):[cite: 45]

Ventaja	Descripción Técnica	Impacto en la Administración
Reutilización y Mantenimiento	Encapsula la lógica de negocio y administración. Si un proceso de backup cambia, solo se modifica el objeto en el servidor, no las aplicaciones externas.	Menor duplicación de código y reducción del error humano.
Seguridad de Acceso	Permite restringir el acceso directo a tablas físicas mediante permisos GRANT EXECUTE sobre procedimientos específicos.	El usuario opera la BD a través de "cajas negras" controladas.
Rendimiento de Red	Reduce el tráfico de red enviando únicamente la orden CALL en lugar de miles de sentencias SQL secuenciales.	Ejecuciones mucho más rápidas en redes congestionadas o entornos cloud.

3. Catálogo del Sistema (Metadatos)

MySQL almacena la información sobre su propia estructura (metadatos) en esquemas específicos[cite: 45]. Dominar estos catálogos es la herramienta más potente de un Administrador de Sistemas (SysAdmin), ya que permite auditar el servidor, monitorizar el almacenamiento y diagnosticar la degradación de rendimiento directamente mediante sentencias SQL[cite: 45].

3.1. information_schema (El Diccionario de Datos)

Es una base de datos virtual de solo lectura definida por el estándar ANSI SQL[cite: 45]. Actúa como el diccionario de datos del SGBD[cite: 45]. No contiene archivos de datos físicos; en su lugar, MySQL genera sus tablas dinámicamente en memoria al consultarlas[cite: 45].

- **SCHEMATA:** Almacena información sobre las bases de datos existentes en el servidor (nombres, juegos de caracteres predeterminados, etc.)[cite: 45].
- **TABLES:** Contiene metadatos de todas las tablas físicas y vistas (motor de almacenamiento, número estimado de filas, tamaño de los datos y de los índices)[cite: 45].
- **COLUMNS:** Detalla cada columna de cada tabla (tipos de datos, nulabilidad, privilegios asignados)[cite: 45].
- **STATISTICS:** Proporciona información de bajo nivel sobre los índices de las tablas del sistema[cite: 45].
- **ROUTINES:** Almacena información sobre los procedimientos y funciones del SGBD[cite: 45].

Caso Práctico 1: Obtener el tamaño en disco de cada base de datos[cite: 45]

Esta consulta calcula el espacio real en disco de cada base de datos (Datos + Índices) expresado en Megabytes:[cite: 45]

```
SELECT
table_schema AS 'Base de Datos',
ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Tamaño
total (MB)',
ROUND(SUM(data_length) / 1024 / 1024, 2) AS 'Solo Datos (MB)',
ROUND(SUM(index_length) / 1024 / 1024, 2) AS 'Solo Índices (MB)'
FROM information_schema.tables
GROUP BY table_schema
ORDER BY (SUM(data_length + index_length)) DESC;
```

Caso Práctico 2: Detectar tablas sin Clave Primaria (Primary Key)[cite: 45]

La falta de claves primarias degrada catastróficamente el rendimiento, especialmente en entornos con replicación activa (Master-Slave) o clústeres[cite: 45].

```
SELECT
t.table_schema AS 'Base de Datos',
t.table_name AS 'Tabla sin PK'
FROM information_schema.tables t
LEFT JOIN information_schema.table_constraints tc
```

```
ON t.table_schema = tc.table_schema
AND t.table_name = tc.table_name
AND tc.constraint_type = 'PRIMARY KEY'
WHERE tc.constraint_name IS NULL
AND t.table_schema NOT IN ('information_schema', 'mysql',
'performance_schema', 'sys')
AND t.table_type = 'BASE TABLE';
```

3.2. Base de Datos mysql (El Repositorio de Seguridad y Privilegios)

A diferencia de information_schema, la base de datos mysql contiene tablas físicas persistentes en el disco[cite: 45]. Es el cerebro de la seguridad de accesos del servidor[cite: 45].

- **user:** Almacena los usuarios creados, hosts permitidos, contraseñas cifradas y privilegios globales del sistema[cite: 45].
- **db:** Controla los privilegios específicos de los usuarios sobre bases de datos completas[cite: 45].
- **tables_priv y columns_priv:** Regulan accesos extremadamente granulares a nivel de tabla o columna[cite: 45].

Caso Práctico 3: Auditoría rápida de expiración de contraseñas de usuarios[cite: 45]

Para cumplir con normativas de seguridad (como el RGPD), un SysAdmin debe controlar la antigüedad y vigencia de las cuentas del sistema:[cite: 45]

```
SELECT
user AS 'Usuario',
host AS 'Host de Origen',
password_expired AS '¿Expirada?',
password_last_changed AS 'Último cambio',
password_lifetime AS 'Días de ciclo de vida'
FROM mysql.user
WHERE ssl_type != '' OR password_expired = 'Y' OR user !=
'mysql.infoschema';
```

3.3. Base de Datos performance_schema

Se enfoca de forma estricta en el rendimiento físico del servidor[cite: 45]. Cuenta con una arquitectura basada en **Instrumentos** (código fuente monitorizado) y **Consumidores** (tablas del motor de almacenamiento en memoria que recopilan las métricas)[cite: 45].

- **events_statements_summary_by_digest:** Agrupa estadísticas sobre patrones de consultas (digests), facilitando identificar cuáles consumen más CPU o tiempo de ejecución[cite: 45].
- **threads:** Muestra información técnica detallada sobre los hilos de ejecución activos del servidor[cite: 45].
- **events_errors_summary_by_user_by_error:** Registra resúmenes históricos de los errores de sintaxis o conexión provocados por cada usuario del sistema[cite: 45].

Caso Práctico 4: Listar los errores más comunes provocados por los usuarios[cite: 45]

```
SELECT
user AS 'Usuario',
error_number AS 'Código de Error',
error_name AS 'Nombre de Error',
sum_error_raised AS 'Veces provocado',
first_seen AS 'Primera detección',
last_seen AS 'Último registro'
FROM performance_schema.events_errors_summary_by_user_by_error
WHERE sum_error_raised > 0
ORDER BY sum_error_raised DESC;
```

3.4. Base de Datos sys (La Abstracción Simplificada)

La complejidad de performance_schema dificulta su uso rutinario en el día a día del SysAdmin[cite: 45]. Para solucionar esto, MySQL incluye la base de datos sys, que agrupa vistas estructuradas de forma intuitiva, funciones y procedimientos predefinidos[cite: 45].

Caso Práctico 5: Identificar las consultas SQL que provocan mayor latencia[cite: 45]

```
SELECT
query AS 'Sentencia SQL',
```

```
db AS 'Base de Datos',
exec_count AS 'Total Ejecuciones',
total_latency AS 'Tiempo Total acumulado',
max_latency AS 'Peor latencia registrada'
FROM sys.statement_analysis
ORDER BY total_latency DESC
LIMIT 5;
```

Caso Práctico 6: Encontrar tablas con lecturas completas[cite: 45]

La monitorización sistemática de consultas que no aprovechan los índices evita problemas de degradación del disco físico y ahorra energía en el centro de datos[cite: 45].

```
SELECT
schemaname AS 'Base de Datos',
tablename AS 'Tabla',
scans AS 'Lecturas Completas',
rows_sel AS 'Filas devueltas'
FROM sys.schema_tables_with_full_table_scans
ORDER BY scans DESC;
```

4. Automatización Mediante Scripts Externos (Bash)

La verdadera automatización de la administración ocurre cuando conectamos el SGBD con el Sistema Operativo utilizando scripts de consola (por ejemplo, Bash en sistemas Linux)[cite: 45].

Para que un script de Bash interactúe con MySQL de forma desasistida, solemos utilizar el cliente de línea de comandos de MySQL mediante la estructura:[cite: 45]

mysql -u "usuario" -p"contraseña" -e "CONSULTA_SQL"

Buenas Prácticas de Seguridad: Exponer la contraseña directamente en el script utilizando -p"mi_password" es una vulnerabilidad grave (cualquier usuario del sistema podría verla ejecutando ps aux)[cite: 45].

Recomendación: Guardar las credenciales en un archivo `.env` con permisos 600 y cargarlo desde el propio script con `source .env`[cite: 45]. De este modo se evita dejar la contraseña en el historial del sistema o en el código fuente[cite: 45].

```
# .env (guardar en la carpeta del proyecto y protegerlo: chmod 600 .env)
MYSQL_USER=root
MYSQL_PASSWORD='PasswordMuySegura123!'
MYSQL_DATABASE=tienda_informatica
BACKUP_DIR="/var/backups/mysql"
```

5. Ejemplos Prácticos de Scripts

Para asentar estos conceptos, a continuación se presentan tres scripts sencillos para administración de sistemas Linux/MySQL[cite: 45]. Están diseñados bajo criterios didácticos, con un nivel de complejidad progresivo y comentarios exhaustivos de cada instrucción[cite: 45].

Script 1: Comprobación de Salud del Servidor (Health Check)[cite: 45]

Este script verifica si el servicio MySQL está activo en el sistema operativo y si responde a consultas básicas[cite: 45]. Si falla, genera una alerta visual y un registro en el log del sistema[cite: 45].

```
#!/bin/bash
set -euo pipefail

# Cargar variables seguras desde .env
if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] No existe el archivo .env. Crea uno con MYSQL_USER y MYSQL_PASSWORD." >&2
  exit 1
fi

# Seguridad: evitar que el archivo .env sea visible por otros usuarios
umask 077

FECHA_LOG=$(date "+%Y-%m-%d %H:%M:%S")
```

```
echo "=== INICIANDO CONTROL DE SALUD DEL SGBD ($FECHA_LOG) ==="

if ! systemctl is-active --quiet mysql; then
  echo "[ALERTA] MySQL está caído. Intentando arrancarlo..."
  sudo systemctl start mysql
  sleep 3
fi

if MYSQL_PWD="$MYSQL_PASSWORD" mysqladmin --user="$MYSQL_USER" ping &>/dev/null; then
  echo "[OK] MySQL está levantado y responde correctamente."
else
  echo "[CRÍTICO] MySQL está activo pero no acepta conexiones."
  exit 1
fi

echo "=== CONTROL DE SALUD FINALIZADO ==="
exit 0
```

Script 2: Copia de Seguridad Sencilla (Backup Automatizado)[cite: 45]

Genera un respaldo (.sql) de una base de datos específica, le asigna una marca de tiempo para no sobreescribir copias anteriores y comprime el archivo para ahorrar espacio en disco[cite: 45].

```
#!/bin/bash
set -euo pipefail

if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] Falta el archivo .env" >&2
  exit 1
fi

mkdir -p "$BACKUP_DIR"
chmod 700 "$BACKUP_DIR"

MARCA_TIEMPO=$(date "+%Y%m%d_%H%M%S")
FICHERO_BACKUP="$BACKUP_DIR/backup_${MYSQL_DATABASE}_${MARCA_TIEMPO}.sql"

echo "Iniciando copia de seguridad de $MYSQL_DATABASE..."
```

```
MYSQL_PWD="$MYSQL_PASSWORD" mysqldump \
  --user="$MYSQL_USER" \
  --databases "$MYSQL_DATABASE" \
  --single-transaction \
  --routines \
  --events \
  > "$FICHERO_BACKUP"

gzip -f "$FICHERO_BACKUP"

echo "Copia creada correctamente en ${FICHERO_BACKUP}.gz"
exit 0
```

Script 3: Comprobación de Consumo y Limpieza de Logs[cite: 45]

Un administrador debe controlar el tamaño de la base de datos[cite: 45]. Este script consulta el catálogo information_schema para verificar si el tamaño supera un límite seguro y limpia una tabla histórica si es necesario para liberar espacio[cite: 45].

```
#!/bin/bash
set -euo pipefail

if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] Falta el archivo .env" >&2
  exit 1
fi

LIMITE_MB=500
QUERY_TAMAÑO="SELECT ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) FROM informatio

echo "Consultando el tamaño total ocupado en disco por el SGBD..."
TAMANO_ACTUAL=$(MYSQL_PWD="$MYSQL_PASSWORD" mysql --user="$MYSQL_USER" -AN -e "$QUERY_TAMAÑO
TAMANO_ENTERO=${TAMANO_ACTUAL%.*}

echo "Tamaño de datos actual en el servidor: $TAMANO_ACTUAL MB"

if [ "$TAMANO_ENTERO" -gt "$LIMITE_MB" ]; then
  echo "[AVISO] Se ha superado el umbral seguro de $LIMITE_MB MB."
  echo "[MANTENIMIENTO] Procediendo a purgar datos antiguos..."
```

```
  QUERY_PURGA="DELETE FROM logs_sistema.registro_eventos WHERE fecha_evento < NOW() - INTERV
  MYSQL_PWD="$MYSQL_PASSWORD" mysql --user="$MYSQL_USER" -e "$QUERY_PURGA"

  echo "Purga de registros completada."
else
  echo "[INFORMACIÓN] El espacio ocupado está dentro de los rangos óptimos."
fi

exit 0
```